期中复习题库（第 1—6 章）·纯题目版

目录

第 1 章 概论 **3**
一、填空题 3
二、选择题 3
三、辨析与简答 3
四、算法设计与分析 4
五、数学证明 4

第 2 章 线性表 **5**
一、填空题 5
二、选择题 5
三、辨析与简答 6
四、算法设计与分析 6
五、数学证明 6

第 3 章 栈与队列 **9**
一、填空题 9
二、选择题 9
三、辨析与简答 9
四、算法设计与分析 10
五、数学证明 11

第 4 章 字符串 **13**
一、填空题 13
二、选择题 13
三、辨析与简答 13
四、算法设计与分析 14
五、数学证明 14

第 5 章 二叉树 **15**
一、填空题 15
二、选择题 15
三、辨析与简答 16
四、算法设计与分析 16
五、数学证明 18

第 6 章 树 **19**
一、填空题 19
二、选择题 19
三、辨析与简答 20

四、算法设计与分析 20
五、数学证明 21

模拟卷（120 分钟，100 分） **23**
一、填空题（每题 2 分，第 1 题两空各 1 分，共 20 分） 23
二、选择题（每题 2 分，共 16 分） 23
三、辨析与简答题（每题 6 分，共 24 分） 23
四、算法设计与分析题（每题 10 分，共 30 分） 24
五、证明题（10 分） 24

本篇按《数据结构与算法》期中复习提要逐条配题，范围是第 1 章到第 6 章。提要里标 ★ 的条目 是重中之重，题号后同样标 ★；没标的也配了题，但分量轻一些。**提要原件逐字存在仓库里**：`ref_ 复习大纲/期中复习提要.md`。

怎么用：这是去掉全部答案与考前速查表的纯题目版，可直接当卷子做；答案与讲解见完整版《期中复习题库》，题号两版一致。每章按考试题型分五组：填空、选择、辨析与简答、数据结构或算法的设计与分析、数学证明，分章题共 167 道；之后是一套完整的模拟卷（26 道，按真实考试的分值和时间配比出的）。

几个贯穿全篇的约定（考试写答案时最好也先声明一句，避免歧义）：

- 数组、字符串、树的层次编号一律**从 0 开始**；根结点在**第 0 层**。凡是与此不同的题，题面会单独说明。
- 「深度为k 的二叉树」指最大层号为k，此时结点数至多$2^{k+1}-1$；树的「高度」默认指**层数**（若按边数计会单独说明）；「n 个结点的完全二叉树高度」按提要口径同样指**层数**，即$\lceil \log_2(n+1) \rceil$。
- 算法设计题按提要的要求：**写明基本思想、无歧义即可**，必要时加注释。用到栈、队列等现成结构时，先写出 ADT（函数名、入口参数、出口参数、功能一句话），再在算法里调用。本篇提供了常用 ADT 的写法，后续题目可引用相应接口。
- 本篇的「满二叉树／满K 叉树」采用严格（正则）树的含义：每个内部结点恰有 2 ／ K 个孩子，不要求叶结点同层；所有层都填满时另称「各层全满」。K、k 出现在除以$K-1$、$k-1$ 的公式中时均要求至少为 2。
- 除另有说明，结点数据的比较、赋值和交换按常数时间计；树的遍历序列用于还原结构时，结点标识互不相同。
- 空指针计数公式均针对非空树，且只计结点内的孩子指针域。
- 伪代码用缩进表示层次，← 是赋值，= 是比较；逻辑运算采用短路求值。

第 1 章 概论

一、填空题

1-1. 数据结构的研究内容包括三个方面：____、____ 和 ____。

1-2. 抽象数据类型（ADT）由 ____ 和 ____ 两部分组成，它**不包括** ____。

1-3. 数据的四种基本存储方式是 ____、____、____、____。

1-4. 大O 表示法的定义：称$f(n) = O(g(n))$，当且仅当存在正常数 ____ 和 ____，使得对所有 ____ 都有 ____ 成立。

1-5. 把下列函数按渐进增长率从小到大排序：$n\log n$，2^n，n^2，$\log n$，$n!$，n，n^3，1，$\sqrt{n}$，2^{2^n}。

1-6. 本书列出的算法基本性质有 ____、____、____、____。

1-7. 数据的逻辑结构用二元组(K, R) 表示时，K 是 ____，R 是 ____；画结构图时，R 中的序偶$\langle x, y\rangle$ 必须画成 ____。

二、选择题

1-8. 下列关于抽象数据类型的叙述，**错误**的是（　）。A. ADT 给出了数据对象、数据关系和基本运算 B. ADT 的定义中包含了数据的存储结构 C. 同一个 ADT 可以有多种不同的实现 D. ADT 把「做什么」和「怎么做」分开

1-9. 以下说法正确的是（　）。A. 算法的时间代价必定随问题规模单调增长 B. 若$f(n) = O(n^2)$，则$f(n)$ 一定不是$O(n)$ C. 大O 给出的是渐进上界，同一个函数可以有多个正确的大O 表达式 D. 最坏情况分析和大O 分析是同一件事

1-10. 下面程序段中循环体执行次数的渐进紧确界是（　）。

```
s ← 0;  i ← 1
do
    s ← s + 5 * i
    i ← i + 1
while (s < n) and (i < n)
```

A. $\Theta(n)$　　B. $\Theta(\sqrt{n})$　　C. $\Theta(\log n)$　　D. $\Theta(n^2)$

1-11. 对同一问题，算法 A 的时间代价为$1000n$，算法 B 为n^2。下列说法正确的是（　）。A. A 总比 B 快　　B. B 总比 A 快　　C. $n > 1000$ 时 A 比 B 快　　D. 两者渐进同阶

1-12. 下列各式中**不成立**的是（　）。A. $3n^2 + 10n = O(n^2)$　　B. $n^2/(n+1) = O(n)$　　C. $2^{n+1} = O(2^n)$　　D. $2^{2n} = O(2^n)$

三、辨析与简答

1-13. 辨析：「数据的逻辑结构一旦确定，存储结构也就确定了。」

1-14. ★ 简答：最坏情况分析与大O表示法有什么区别和联系？为什么说「某算法的时间复杂度是$O(n^2)$」这句话本身并不说明它是最坏情况？

1-15. 简答：时间与空间的折衷，举一个例子说明。

1-16. 简答：为什么渐进分析要「保留增长最快的那一项」并丢掉常数因子？什么时候这样做会误导人？

四、算法设计与分析

1-17. 设$K = \{k_1, k_2, k_3, k_4\}$，$R = \{\langle k_1, k_2\rangle, \langle k_2, k_3\rangle, \langle k_2, k_4\rangle, \langle k_4, k_1\rangle\}$。画出该逻辑结构的示意图，并说明它属于哪一类结构。

1-18. ★ 设计：某系统要维护一批学生成绩，要求支持三种操作—— (a) 按学号查一个学生 (b) 插入一个新学生 (c) 按成绩从高到低输出全部学生。若三种操作的频率相当，请给出一种数据结构设计并分析代价；若 (c) 只在学期末执行一次，设计又该怎么改？

1-19. ★ 分析下列程序段中 `x ← x + 1` 的执行次数m，并给出时间复杂度。

```
for i ← 1 to n
    for j ← 1 to n - 2 * i + 1
        x ← x + 1
```

1-20. ★ 分析下面程序段中内层循环体的执行次数，给出时间复杂度。

```
for i ← 1 to n
    j ← i
    while j ⩽ n
        操作
        j ← j + i
```

1-21. ★ 求下列递推式的渐进解：(1) $T(n) = T(n/2) + c$；(2) $T(n) = 2T(n/2) + c$；(3) $T(n) = 2T(n/2) + n$；(4) $T(n) = T(n-1) + n$。

五、数学证明

1-22. 用大O的定义证明$3n^2 + 10n + 7 = O(n^2)$，并给出一组具体的c与n_0。

1-23. 证明：对任意非负函数f, g，有$\max\{f(n), g(n)\} = \Theta(f(n) + g(n))$。

1-24. ★ 证明$T(n) = 2T(\lfloor n/2\rfloor) + n$（$T(1) = 1$）的解是$\Theta(n \log n)$。

1-25. 设f, g为非负函数（(1) 中取对数时要求$f, g > 0$），判断下列命题真假并证明或举反例：(1) 若$f(n) = O(g(n))$，则$\log f(n) = O(\log g(n))$；(2) 若$f(n) = O(g(n))$，则$2^{f(n)} = O(2^{g(n)})$；(3) 若$f(n) = O(g(n))$，则$f(n)^2 = O(g(n)^2)$。

1-26. 证明$\log_2(n!) = \Theta(n \log n)$。

第 2 章 线性表

一、填空题

2-1. 线性表是n（$n \geq 0$）个数据元素的 ____ 序列。表中除第一个元素外，每个元素有且仅有 一个 ____；除最后一个元素外，每个元素有且仅有一个 ____。

2-2. 长度为n 的顺序表，在等概率前提下，插入一个元素平均需要移动 ____ 个元素，删除一个元素平均需要移动 ____ 个元素。

2-3. 在带头结点的单链表 `L` 中，空表的判定条件是 ____；在不带头结点的单链表中，空表的判定条件是 ____。

2-4. 在双链表中删除指针 `p` 所指的结点（`p` 不是首尾结点），需要执行的两条指针操作是 ____ 和 ____。

2-5. 设每个数据域大小为s、指针大小为p（忽略头结点、内存对齐及分配开销）。当表的装填程度（实际元素数 / 预分配容量）小于 ____ 时，链表比顺序表省空间。

2-6. 在循环单链表中，判断指针 `p` 指向尾结点的条件是 ____。

二、选择题

2-7. 在一个长度为n 的顺序表中，在第i 个元素之前插入一个新元素（$1 \leq i \leq n+1$），需要向后移动 ____ 个元素。A. $n-i$　B. $n-i+1$　C. $n-i-1$　D. i

2-8. ★ 已知 `p` 指向带头结点单链表中的某个结点（非尾结点），要在 `p` 之后插入结点 `s`，下列语句序列正确的是（　）。A. `s->next = p->next; p->next = s;` B. `p->next = s; s->next = p->next;` C. `s->next = p; p->next = s;` D. `p->next = s->next; s->next = p;`

2-9. ★ 设 `p` 指向单链表中的某结点，下列操作**不能**在$O(1)$ 时间内完成的是（　）。A. 删除 `p` 的后继结点 B. 在 `p` 之后插入一个结点 C. 删除 `p` 本身（单链表，仅给 `p`）D. 访问 `p` 的数据域

2-10. 下列关于顺序表与链表的比较，**正确**的是（　）。A. 顺序表查找任意元素都比链表快 B. 链表的插入删除一定比顺序表快 C. 顺序表适合表长稳定、以随机访问为主的场合 D. 链表不需要预先估计表长，所以任何情况下都更优

2-11. 带头结点的循环双链表 `L` 为空的判定条件是（　）。A. `L == nullptr`　B. `L->next == nullptr` C. `L->next == L`　D. `L->prev == nullptr`

2-12. ★ 在带头结点的循环双链表中，`p` 指向某个结点，要在 `p` 之后插入结点 `s`。下列语句序列中**错误**的是（　）。A. `s->prev = p; s->next = p->next; p->next->prev = s; p->next = s;` B. `s->next = p->next; p->next->prev = s; s->prev = p; p->next = s;` C. `s->prev = p; s->next = p->next; p->next = s; p->next->prev = s;` D. `s->next = p->next; s->prev = p; p->next = s; s->next->prev = s;`

三、辨析与简答

2-13. 辨析：「链表比顺序表节省存储空间。」

2-14. 简答：头结点（附加表头结点）有什么作用？不带头结点会多出哪些特例？

2-15. ★ 简答：从**查找、插入删除、空间、适用场合**四个方面比较顺序表和链表。

四、算法设计与分析

本组题统一使用如下单链表 ADT（考试时也建议先写这一段，再写算法）：

```
结点 Node
    data    元素值
    next    指向下一结点的指针，尾结点为 nullptr

链表 List（带头结点，head 指向头结点）
    Head()              出口：头结点指针
    Empty()             出口：表空返回真
    MakeNode(x)         入口：元素值 x；出口：新结点指针，data 为 x、next 为空
    FreeNode(p)         入口：结点指针 p；功能：释放该结点
```

2-16. ★ 设计算法把带头结点的单链表**就地逆置**（不申请新结点），并分析时空代价。

2-17. ★ 已知两个带头结点的**递增有序**单链表 A、B，设计算法把它们归并成一个递增有序表 C，要求**不申请新结点**，且结果中相同的值只保留一个。

2-18. 设计算法在带头结点的单链表中查找**倒数第k个**结点，要求只遍历一趟、空间$O(1)$。

2-19. 设计算法判断一个单链表中是否存在环，若存在则返回入环的第一个结点。要求空间$O(1)$。

2-20. ★ 已知 `p` 指向单链表中的某个结点（不是头结点也不是尾结点），要求在$O(1)$ 时间内 删除 `p`。给出算法，并说明它为什么对尾结点无效。

2-21. ★ 双链表中结点 `p` 的前驱和后继都存在，且它的后继不是尾结点。设计算法把 `p` 与它的**后继**交换位置，要求只改指针、不动数据域；并说明为什么这些语句的先后次序不能随意调换。

2-22. 设计算法删除顺序表中所有值等于x 的元素，要求时间$O(n)$、空间$O(1)$。

五、数学证明

2-23. 证明：在长度为n 的顺序表中等概率地插入一个元素，平均移动元素数为$n/2$。

2-24. 证明：对有序的单链表，二分检索无法达到$O(\log n)$。

2-25. 设每个数据元素占s 个存储单元、每个指针占p 个单元，顺序表预分配容量为m、实际存n 个元素。忽略头结点、内存对齐及分配开销，求链表比顺序表省空间的条件。

第 3 章 栈与队列

一、填空题

3-1. 栈是限定只能在 ____ 进行插入和删除的线性表，其特点是 ____；队列是限定在 ____ 插入、在 ____ 删除的线性表，其特点是 ____。

3-2. ★ 设循环队列的数组容量为M（牺牲一格，最多存$M-1$ 个元素），`front` 指向队头元素、`rear` 指向队尾元素的下一个位置。则：队空的条件是 ____，队满的条件是 ____，队中元素个数为 ____。

3-3. ★ 设有n 个元素按$1, 2, \cdots, n$ 的顺序依次入栈（中途可任意出栈），则可能得到的不同出栈序列共有 ____ 种。

3-4. 顺序栈用数组 `data[0..m-1]` 存储，若约定 `top` 指向**栈顶元素**，则空栈时 `top` 为 ____，入栈的两步是 ____；若约定 `top` 指向**栈顶元素的上一个空位**，则空栈时 `top` 为 ____。

3-5. ★ 递归调用时，系统运行栈中每个活动记录（栈帧）主要保存三样东西：____、____、____。

3-6. 中缀表达式 `A + B * (C - D) / E` 的后缀形式是 ____。

二、选择题

3-7. ★ 设入栈序列为 `1,2,3,4`，下列**不可能**是出栈序列的是（　）。A. `1,2,3,4`　B. `4,3,2,1`　C. `3,1,2,4`　D. `2,4,3,1`

3-8. 容量为M 的循环队列（牺牲一格），`front = 3`、`rear = 0`、$M = 8$，则队中元素个数是（　）。A. 3　B. 5　C. 0　D. 8

3-9. 下列关于循环队列的叙述正确的是（　）。A. 循环队列解决的是队列「真溢出」问题 B. 循环队列必须牺牲一个存储单元，没有别的办法 C. 循环队列解决的是「假溢出」问题，即队尾到达数组末端而前端仍有空闲 D. 循环队列中 `front == rear` 一定表示队满

3-10. 后缀表达式 `3 4 + 5 2 - *` 的值是（　）。A. 21　B. 17　C. 35　D. 9

3-11. 用两个栈S_1、S_2 模拟一个队列，下列说法正确的是（　）。A. 每次出队都要把S_1 全部倒进S_2，故出队一定是$O(n)$ B. 入队$O(1)$，出队的**摊还**代价是$O(1)$ C. 无法用两个栈模拟队列 D. 需要三个栈才能模拟

三、辨析与简答

3-12. 辨析：「栈和队列都是线性表，所以它们的存储结构也只能和线性表一样。」

3-13. ★ 辨析：「链式栈没有容量上限、不会上溢，所以总是优于顺序栈。」从时间和空间两方面比较二者。

3-14. ★ 简答：为什么循环队列中 `front == rear` 既可能表示空、也可能表示满？列举至少两种区分办法并比较。

3-15. ★ 简答：什么是递归？用栈消除递归的一般方法是什么？哪些递归可以不用栈就改成循环？

3-16. 简答：递归改成非递归以后，代价一定变小吗？

四、算法设计与分析

3-17. ★ 写出栈和队列的 ADT，并给出顺序栈 `Push`、`Pop` 的实现（含溢出处理）。

3-18. ★ 写出循环队列的完整实现（判空、判满、入队、出队、求长度），采用「牺牲一格」方案。

3-19. ★ 设入栈序列固定为$1, 2, \cdots, n$，给定一个$1..n$ 的排列p，设计算法判断p是否为一个合法的出栈序列。分析代价。

3-20. ★ 写出中缀表达式转后缀表达式的算法，并用它把 `A + B * C - (D / E + F) * G` 转成后缀式。

3-21. 写出后缀表达式求值算法，并求 `A B C * + D E / F + G * -` 在 `A=2, B=3, C=4, D=8, E=2, F=1, G=3` 时的值。

3-22. ★ 用栈把下面的递归算法改写成非递归算法，并说明栈中保存了什么。

```
Hanoi(n, A, B, C)          // 把 n 个盘子从 A 经 B 移到 C
    if n = 0 then return
    Hanoi(n - 1, A, C, B)
    输出 "A → C"
    Hanoi(n - 1, B, A, C)
```

3-23. 设计算法用**一个数组**实现两个栈，使得只有在整个数组真正用完时才发生上溢。

3-24. 设计算法判断一个字符序列是否为回文（正读反读相同），要求用栈和队列各一个。

3-25. ★ **算法填空**：用两个队列实现一个栈。队列只提供 `push(x)` 入队、`pop()` 队首出队、`front()` 取队首、`empty()` 判空（**没有求长度的运算**）。下面这一版让 `push` 保持$O(1)$，把代价放在 `pop` 上；调用 `pop` 的前提是栈非空。补全五个空。

```
class MyStack
    queue q1, q2                        // 元素平时都在 q1 里

    push(x)
        ____(1)____

    pop()                               // 弹出最后压入的那个
        while ____(2)____
            t ← q1.front();  q1.pop()
            if ____(3)____  then  r ← t        // 走到最后一个，它就是答案
            else                  q2.push(t)
        ____(4)____                     // 让 q1 重新成为存元素的那个
```

```
        return r

    empty()
        return ____(5)____
```

五、数学证明

3-26. ★ 证明：n 个元素按$1, 2, \cdots, n$ 顺序入栈，可能的出栈序列共$C_n = \frac{1}{n+1}C_{2n}^{n}$ 种。

3-27. ★ 证明：排列$p_1 p_2 \cdots p_n$ 是入栈序列$1, 2, \cdots, n$ 的合法出栈序列，当且仅当**不存在下标**$i < j < k$ **使**$p_j < p_k < p_i$。

3-28. 证明循环队列中「元素个数$= (rear - front + M) \bmod M$」对 `rear` 是否绕回都成立。

3-29. ★ 用 `I` 表示一次入栈、`O` 表示一次出栈。一个从空栈开始、到空栈结束的操作过程可以写成一个由 `I`、`O` 组成的序列（例如 `IOIO` 合法，`IOOI` 非法）。(1) 给出序列合法的**充要条件**并说明理由；(2) 证明：对元素互不相同的输入，两个**不同的**合法操作序列不可能产生**相同的**输出序列；(3) 长度为$2n$ 的合法序列共有多少个？

第 4 章 字符串

一、填空题

4-1. 字符串是由 ____ 组成的有限序列，串中字符的个数称为串的 ____。不含任何字符的串称为 ____，它的长度为 ____；由一个或多个空格组成的串称为 ____，它的长度 ____。

4-2. 串 `S = "abcaabcab"` 的长度是 ____，它的所有**真前缀**中最长且同时也是真后缀的那一个是 ____。

4-3. 设目标串长n、模式串长m。朴素（Brute Force）模式匹配算法的最坏时间代价是 ____，KMP 算法的时间代价是 ____，其中求特征向量需 ____、匹配过程需 ____。

4-4. ★ 模式 `P = "abaabcaba"`（下标从 0）的**未优化**特征向量（next 数组，约定 `next[0] = -1`）是 ____。

4-5. ★ 模式 `P = "ababab"`（下标从 0）的**未优化** next 数组是 ____，**优化后**的 next 数组是 ____。

4-6. ★ 设非空串S 长n，其**整串**的最长公共真前后缀长度为f。则S 的最小循环节长度是 ____；S 由该循环节至少重复两次构成的充要条件是 ____。

二、选择题

4-7. 下列关于串的叙述**错误**的是（　）。A. 空串是任意串的子串 B. 空格串的长度等于其中空格的个数 C. 长度为n 的串，按起止位置计数（相同内容重复计）的非空子串个数为$\frac{n(n+1)}{2}$ D. 两个串相等，只要求它们包含的字符集合相同

4-8. ★ 模式 `P = "aaab"` 的未优化 next 数组是（　）。A. `-1 0 1 2`　B. `-1 0 0 0`　C. `0 1 2 3`　D. `-1 1 2 3`

4-9. ★ 用 KMP 匹配时，若模式在位置i 处失配且 `next[i] = -1`，则下一步应当（　）。A. 模式右移一位，目标下标回退到本趟起点加一 B. 模式整体右移$i+1$ 位，目标下标前进一格，从 `P[0]` 重新比 C. 目标下标回退到0 D. 匹配失败，算法结束

4-10. 用朴素算法在T = `"aaaaaaaaab"`（9 个 `a` 加 1 个 `b`，长 10）中匹配P = `"aaab"`（长 4），字符比较的总次数是（　）。A. 10　B. 24　C. 28　D. 40

三、辨析与简答

4-11. 辨析：「串是一种特殊的线性表，所以线性表的所有算法都能直接用在串上。」

4-12. ★ 简答：KMP 算法为什么能做到$O(n+m)$？它省掉的究竟是什么？

4-13. ★ 简答：未优化的 next 与优化后的 next 有什么区别？为什么优化后会出现-1？

4-14. 简答：原书的朴素匹配算法返回值有一处差 1 的错误，说说这类错误为什

么难被发现。

4-15. 简答：实现 `my_memcpy(void* dst, const void* src, size_t n)` 时，若两块内存**重叠**会发生什么？分 `src < dst < src + n` 与 `dst < src < dst + n` 两种情形说明正确做法。

4-16. ★ 设 `S = "abaabab"`（下标从 0）。(1) 按字典序列出 `S` 的全部**不同**非空子串，并说明共有多少个；(2) 求 `S` 的未优化 next 数组和优化后的 next 数组。

四、算法设计与分析

4-17. ★ 写出求未优化特征向量（next 数组）的算法，并说明其时间代价。

4-18. ★ 写出 KMP 匹配算法（利用已求出的 next 向量），并分析其代价。

4-19. ★ 对模式 `P = "abaabcac"`，求它的未优化 next 与优化 next；并用 KMP 在$T =$ `"abaabaabcac"` 中匹配，写出每一趟的失配位置与右移位数。

4-20. ★ 变种应用：给定串S（长n），设计$O(n)$ 算法求它的**最小循环节**，并判断S 是否由该循环节至少重复两次构成。

4-21. 变种应用：设计算法判断串B 是否为串A 的**循环移位**（如`"cdeab"` 是`"abcde"` 的循环移位），要求$O(|A| + |B|)$。

4-22. 变种应用：若某个子串形如AA（非空串A 紧接着再出现一次，如`"bcbc"`、`"aa"`），称它为**平方子串**。设计算法判断串S（长n）是否含平方子串；若含，返回**起点最小**的那个的起点和$|A|$。要求最坏$O(n^2)$。

五、数学证明

4-23. ★ 证明 KMP 匹配过程的时间代价为$O(n)$（n 为目标串长度）。

4-24. ★ 设非空串S 长n，f 是S 的最长公共真前后缀长度。证明：$p = n - f$ 是S 的最小周期，且S 能写成某个串至少重复两次（$S = w^k$，$k \geq 2$）当且仅当$f > 0$ 且p 整除n。

4-25. 证明：求 next 数组的算法（4-17）时间代价为$O(m)$。

第 5 章 二叉树

一、填空题

5-1. 二叉树是结点的有限集合，它或者为空，或者由一个根结点及两棵**互不相交**的、分别称为 ____ 和 ____ 的二叉树组成。含 3 个结点的二叉树共有 ____ 种不同形态。

5-2. 二叉树的第i 层（根为第 0 层）最多有 ____ 个结点；深度为k 的二叉树最多有 ____ 个结点；有n 个结点的完全二叉树，其高度（层数）为 ____。

5-3. ★ 在任意一棵非空二叉树中，若叶结点数为n_0、度为 2 的结点数为n_2，则它们的关系是 ____。若该树共有n 个结点、度为 1 的结点n_1 个，则$n =$ ____。

5-4. 完全二叉树按层次顺序从 0 开始编号存入一维数组，则结点i 的父结点编号是 ____，左孩子是 ____，右孩子是 ____；结点i 有左孩子的条件是 ____。

5-5. 有n（$n \geq 2$）个结点的完全二叉树，最后一个分支（非叶）结点的编号是 ____，叶结点共有 ____ 个。

5-6. 某二叉树的前序序列为 `ABDECFG`，中序序列为 `DBEAFCG`，则它的后序序列为 ____，层次序列为 ____。

5-7. ★ 含n 个叶结点（权值互不相同）的 Huffman 树共有 ____ 个结点，其中度为 2 的结点 ____ 个，**没有**度为 ____ 的结点。

5-8. 二叉搜索树的 ____ 遍历序列是一个递增有序序列；在一棵含n 个结点的二叉搜索树中检索一个关键码，最好情况比较 ____ 次，最坏 ____ 次。

二、选择题

5-9. 一棵完全二叉树有 1001 个结点，其中叶结点的个数是（ ）。A. 250　B. 500　C. 501　D. 505

5-10. 已知二叉树的**前序**和**后序**序列，下列说法正确的是（ ）。A. 能唯一确定这棵二叉树 B. 不能唯一确定，但能确定它的中序序列 C. 不能唯一确定这棵二叉树 D. 只有当树中没有度为 1 的结点时才不能确定

5-11. ★ 在下列关键码序列中，可以构成**最小堆**的是（ ）。A. `(12, 70, 33, 65, 24, 56)`　B. `(12, 24, 33, 65, 70, 56)` C. `(70, 65, 56, 33, 24, 12)`　D. `(12, 24, 33, 65, 70, 30)`

5-12. 在二叉搜索树中删除一个既有左子树又有右子树的结点，下列做法**能**保持二叉搜索树性质的是（ ）。A. 用它的左孩子顶替 B. 用它的右孩子顶替 C. 用它在中序序列中的前驱（左子树中最大者）顶替 D. 直接置空

5-13. 关于 Huffman 编码，下列说法**错误**的是（ ）。A. 它是一种前缀编码，任何字符的编码都不是另一个字符编码的前缀 B. 权值较大的字符，其码长不会比权值较小的字符更长 C. Huffman 树的形态可能不唯一，但带权路径长度 WPL 一定唯一 D. Huffman 编码是定长编码

5-14. 用筛选法对n 个元素的无序序列建最小堆，时间代价的渐进紧确界是（　）。
A. $\Theta(n)$　　B. $\Theta(n\log n)$　　C. $\Theta(\log n)$　　D. $\Theta(n^2)$

三、辨析与简答

5-15. 辨析：「二叉树就是度不超过 2 的树。」

5-16. ★ 简答：为什么「前序 + 中序」或「后序 + 中序」能唯一确定一棵二叉树，而「前序 + 后序」不能？

5-17. ★ 某二叉树的 9 个结点标识为 `A`～`I`，三种遍历序列都有残缺（`_` 表示缺失的一位）：

```
前序  A B _ E G _ F _ I
中序  D _ G E A C H _ I
后序  _ G E B H _ F C A
```

补全三个序列，并画出这棵二叉树。

5-18. ★ 简答：堆和二叉搜索树都是二叉树，它们的「有序性」有什么本质区别？各适合什么场合？

5-19. 简答：非递归的二叉树遍历为什么需要栈？层次遍历为什么用队列？

四、算法设计与分析

本组题统一使用如下二叉树 ADT：

```
结点 Node
    data          结点数据
    left, right   左右孩子指针，空为 nullptr

BinaryTree
    Root()                出口：根指针
    Left(p) / Right(p)    入口：结点指针；出口：左/右孩子指针
    Value(p)              入口：结点指针；出口：该结点数据
```

5-20. ★ 用**递归**写出下列算法，并分析代价：(1) 求二叉树的高度；(2) 求叶结点个数；(3) 交换所有结点的左右子树（求镜像）；(4) 判断两棵二叉树是否相同。

5-21. ★ **算法填空**：下面是二叉树**中序**遍历的非递归框架，补全空缺处。

```
InOrder(root)
    S ← 空栈
    p ← root
    while p ≠ nullptr or not S.Empty()
        if p ≠ nullptr
            S.Push(p)
            p ← ____(①)____
```

```
    else
        p ← S.Pop()
        访问 p
        p ← ____(②)____
结束
```

5-22. ★ 给定关键码序列$K = \{50, 19, 35, 55, 20, 5, 100, 52, 88, 53, 92\}$, (1) 按此次序依次插入, 画出生成的二叉搜索树; (2) 写出中序序列; (3) 删除结点 50, 画出删除后的树并说明步骤。

5-23. ★ 用值为1 到5 的 5 个结点建二叉搜索树。(1) 若要求树上每个结点的**左、右子树结点数之差不超过 1**, 共有多少棵? (2) 若把要求换成每个结点的**左子树结点数不少于右子树**, 又有多少棵?

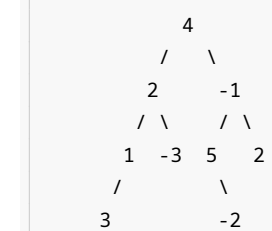
5-24. ★ 已知字符集$\{a, b, c, d, e, f, g, h\}$ 的使用频率依次为 5、29、7、8、14、23、3、11（合计 100）。(1) 构造 Huffman 树; (2) 给出各字符的 Huffman 编码; (3) 求带权路径长度 WPL 与平均码长; (4) 对码流 `000110110` 译码。

5-25. ★ 字符集$\{A, B, C, D\}$ 的出现频率为$\{50\%, 25\%, 12.5\%, 12.5\%\}$。(1) 构造 Huffman 树, 给出各字符的编码与平均码长, 并与定长编码比较; (2) 若D 的频率**上升**、其余三个字符的相对比例不变, 而**编码方式仍用第 (1) 问算出的那一套**, D 的频率升到多少时, 这套 Huffman 编码反而不如定长编码省?

5-26. ★ 给定序列$K = \{53, 17, 78, 9, 45, 65, 87, 23, 31\}$, (1) 用筛选法把它建成**最小堆**, 写出每一步; (2) 在结果堆上删除最小元, 写出调整后的数组; (3) 回到 (1) 的建堆结果, 插入元素 20, 写出 SiftUp 调整后的数组。

5-27. ★ 一棵N 个结点的二叉树用两个一维数组存储: `L[i]`、`R[i]` 分别是结点i 的左、右孩子编号, 为 0 表示空（结点从 1 开始编号, `L[0] = R[0] = 0`, 根的编号已知）。(1) 设计算法由 `L`、`R` 生成父指针数组 `T`, 使 `T[i]` 为结点i 的父结点编号（根记 0）; (2) 设计算法判断结点U 与V 是否在**同一条从根到叶的路径**上（即其中一个是另一个的祖先）。分析两问的代价。

5-28. 二叉树的结点存整数（可正可负）。从某个结点出发、**只沿孩子方向往下走**、在它自己或它的某个后代处结束, 经过的结点序列称为一条**向下路径**（不必从根开始, 也不必在叶结束）。给定整数 `target`, 设计算法统计结点值之和等于 `target` 的向下路径条数, 并分析代价。对下面这棵树、`target = 3`, 答案是多少?



五、数学证明

5-29. ★ 证明:非空二叉树中叶结点数n_0与度为2的结点数n_2满足$n_0 = n_2+1$。

5-30. ★ 证明：用筛选法把n个元素的无序序列建成堆，时间代价为$O(n)$。

5-31. 证明：n个结点的不同二叉树形态共有$C_n = \frac{1}{n+1}C_{2n}^n$种。

5-32. 证明：n个结点的完全二叉树高度（层数）为$\lceil \log_2(n+1) \rceil$。

第 6 章 树

一、填空题

6-1. 树是n（$n \geq 1$）个结点的有限集合。其中有且仅有一个 ___ 结点，其余结点可分为m 个 ___ 的有限集合，每一个又都是一棵树，称为根的 ___。m 棵互不相交的树的集合称为 ___。

6-2. 结点的**度**是指 ___；树的度是指 ___。若一棵树中度为i 的结点有n_i 个（$i = 0, 1, \cdots, m$），则该树的总结点数$n =$ ___，且有$n =$ ___（用度数表示）。

6-3. 一棵树的嵌套括号表示为 `(A(B(E)(F))(C)(D(G(H))))`。该树共有 ___ 个结点，树的度是 ___，叶结点有 ___ 个，度为 1 的结点有 ___ 个。

6-4. ★ 森林转换成二叉树的规则是：___ 作为二叉树的左子树，___ 作为右子树。转换后，二叉树根结点的**右子树**对应原森林中的 ___。

6-5. ★ 按**先根次序**周游森林，等同于按 ___ 次序周游它对应的二叉树；按**后根次序**周游森林，等同于按 ___ 次序周游它对应的二叉树。二叉树的前序、中序、后序三种深度优先遍历中，不能保证与森林的先根或后根任何一种相同的是 ___ 遍历。

6-6. ★ 完全K 叉树按层次从 0 开始编号，则结点i 的第j 个孩子（$1 \leq j \leq K$）编号为 ___，其父结点编号为 ___；第i 层最多有 ___ 个结点；高度为h 层且各层全满的K 叉树共有 ___ 个结点。

6-7. 一棵有n 个结点的K 叉树，采用「每个结点固定K 个孩子指针」的链式存储时，其中空指针的个数为 ___。

6-8. 并查集（等价类）用 ___ 表示法存储，`Find` 操作沿 ___ 找到 ___，两个元素属于同一集合当且仅当 ___。两条常用的优化是 ___ 和 ___。

二、选择题

6-9. 一棵度为 4 的树中，度为 1、2、3、4 的结点分别有 4、2、1、1 个，则叶结点有（ ）个。A. 6　B. 7　C. 8　D. 9

6-10. ★ 森林F 中有 3 棵树，结点数分别为n_1, n_2, n_3。把F 转换成二叉树B 后，B 的根结点的**右子树**上的结点个数是（ ）。A. n_1　B. $n_1 + n_2$　C. $n_2 + n_3$　D. n_3

6-11. 已知一棵树的先根序列为 `ABCDEF`、后根序列为 `CBEFDA`，则下列说法正确的是（ ）。A. `C` 是 `B` 的孩子，`E`、`F` 都是 `D` 的孩子 B. `A` 有三个孩子 C. 该树的度为 3 D. 该树的层次序列为 `ABCDEF`

6-12. ★ 使用**重量权衡合并规则**的并查集（不含路径压缩），n 个元素任意合并后，树的最大高度的渐进紧确界是（ ）。A. $\Theta(1)$　B. $\Theta(\log n)$　C. $\Theta(\sqrt{n})$　D. $\Theta(n)$

6-13. 对同一片森林，下列两个序列一**般不相同**的是（ ）。A. 森林的先根序列 与

对应二叉树的前序序列 B. 森林的后根序列 与 对应二叉树的中序序列 C. 森林的层次序列 与 对应二叉树的层次序列 D. 以上三组都相同

三、辨析与简答

6-14. 辨析：「因为森林和二叉树可以一一对应，所以它们是同一种数据结构。」

6-15. ★ 简答：为什么树（森林）没有「中根遍历」？先根、后根与对应二叉树的前序、中序 为什么会一一对应？

6-16. 简答：比较树的三种链式存储（子结点表、左子/右兄二叉链、父指针表），各自擅长什么运算？

6-17. ★ 并查集的另一种常见存法：数组 `parent[1..n]` 中，非根结点存父结点编号，**根结点存「−（该集合的元素个数）」**。现有 10 个元素：

```
i          1  2  3  4  5  6  7  8  9  10
parent[i]  3  6 -5  6 -1 -4  3  2  7  9
```

(1) 画出它表示的森林，写出各等价类；
(2) 采用**重量权衡合并 + 路径压缩**，依次执行 `Union(10, 8)`、`Union(5, 4)`，写出每一步之后的数组；
(3) 这种存法比 6-21 的「`parent` 加 `weight` 两个数组」好在哪里？

四、算法设计与分析

本组题统一使用「左子/右兄」表示：

```
结点 TNode
    data        结点数据
    child       指向第一个孩子（长子），无孩子为 nullptr
    sibling     指向下一个兄弟，无兄弟为 nullptr
```

6-18. ★ 已知森林F 由两棵树组成：第一棵的根是 `A`，`A` 的孩子依次是 `B`、`C`、`D`，`C` 的孩子是 `E`、`F`；第二棵的根是 `G`，`G` 的孩子是 `H`、`I`，`H` 的孩子是 `J`。(1) 画出F；(2) 写出先根、后根、层次三种序列；(3) 画出F 对应的二叉树，并写出它的前序、中序序列，与 (2) 对照。

6-19. ★ 在「左子/右兄」表示下，写出森林的**先根、后根**周游算法（递归）和**层次**周游算法（用队列），并分析代价。

6-20. ★ **算法填空**：多叉树的**非递归后根周游**。设树另提供操作 `Child(p, k)`：返回结点 `p` 的第k 个孩子（k 从 1 起），不存在时返回 `nullptr`。栈中每条记录是二元组 `(p, k)`，含义是「结点 `p` 接下来该处理它的第k 个孩子」。

```
PostOrder(root)
    if root = nullptr then return
    S ← 空栈;  S.Push((root, 1))
```

```
while not S.Empty()
    (p, k) ← S.Pop()
    c ← Child(p, k)
    if ____(①)____
        访问 p
    else
        S.Push(____(②)____)
        S.Push(____(③)____)
```

(1) 补全空缺；(2)② 与 ③ 两句能否交换次序?

(2) 设一棵树的根 A 有孩子 B、C、D，C 有孩子 E、F，D 有孩子 G。写出访问序列，以及访问 E 时栈中的记录（自底向上）；

(3) 分析代价。

6-21. ★ 写出并查集的 Find 与 Union，要求实现**重量权衡合并**与**路径压缩**，并分析代价。

6-22. ★ 设有 8 个元素$1..8$，依次读入等价偶对$(1,2)$，$(3,4)$，$(5,6)$，$(7,8)$，$(2,4)$，$(6,8)$，$(4,8)$。采用**重量权衡合并 + 路径压缩**，写出每一步之后的父指针数组，并给出最终的等价类。

6-23. ★ 已知一片森林的**带双标记的先根次序**表示，写出还原成「左子/右兄」链式结构的算法，并对 6-18 那片森林给出它的双标记序列。

6-24. 在「左子/右兄」表示下，写出求森林中结点总数与森林高度的算法。

6-25. ★ 一棵树的**镜面映射**指把每个结点的子结点次序反转。输入输出树时常把树转成对应的二叉树，并给只有一个子结点的分支结点补一个虚子结点 $，形成「**伪满二叉树**」(每个分支结点恰有两个子结点)；再对这棵二叉树作**前序遍历**，内部结点标 0、叶结点标 1，虚结点也输出。现给定这样的前序序列，设计算法输出**原树的镜面映射的广度优先（层次）序列**。以 x0 p0 $1 q0 s0 $1 t1 r1 $1 为例，应输出 x r q p t s。

五、数学证明

6-26. ★ 证明：按先根次序周游森林F 得到的序列，等于按前序周游F 对应二叉树$B(F)$ 得到的序列；按后根次序周游F 的序列，等于按中序周游$B(F)$ 的序列。

6-27. 证明：一棵有n 个结点的K 叉树（每结点固定K 个孩子指针），空指针数为$n(K-1)+1$。

6-28. ★ 证明：采用**重量权衡合并规则**（不含路径压缩）的并查集，任何一棵树的高度不超过$\lfloor \log_2 n \rfloor + 1$ 层。

6-29. 证明：树中所有结点的度数之和等于结点数减 1，即$\sum_i i \cdot n_i = n - 1$。

6-30. ★ 满k 叉树指每个结点要么是叶结点、要么恰有k 个孩子。(1) 证明：非空满k 叉树若有L 个叶结点，则内部结点数为$\frac{L-1}{k-1}$；并由此说明L 必须满足「$(k-1)$

整除$(L-1)$」。(2) 证明：高度为h 层（只有根时$h=1$）的满k 叉树，结点数至少$k(h-1)+1$、至多$\frac{k^h-1}{k-1}$，且两个界都能取到。

模拟卷（120 分钟，100 分）

题目覆盖前面的核心知识点，部分题型有所复现，可当作一次完整自测。

一、填空题（每题 2 分，第 1 题两空各 1 分，共 20 分）

1. 抽象数据类型由数据对象、数据关系和 ____ 三部分组成，其中**不包括** ____。
2. 在长度为 100 的顺序表中删除一个元素，等概率下平均需要移动 ____ 个元素。
3. 容量为 10 的循环队列（牺牲一格），`front = 7`、`rear = 2`，队中有 ____ 个元素。
4. 6 个元素依次入栈，可能的出栈序列共 ____ 种。
5. 模式 `P = "abaabab"` 的未优化 next 数组是 ____。
6. 一棵二叉树有 20 个叶结点、15 个度为 1 的结点，则它共有 ____ 个结点。
7. 有 100 个结点的完全二叉树，叶结点有 ____ 个。
8. 对 6 个字符构造 Huffman 树，树中共有 ____ 个结点。
9. 一片森林有 4 棵树、共 30 个结点，转换成二叉树后，根结点的右子树上有 ____ 个结点（设第一棵树有 9 个结点）。
10. 有n 个结点的 3 叉树（每结点固定 3 个孩子指针），空指针共 ____ 个。

二、选择题（每题 2 分，共 16 分）

1. 下列数据结构中，**不属于**线性结构的是（　）。A. 栈　B. 队列　C. 串　D. 二叉树
2. 入栈序列为 `1,2,3,4,5`，下列**不可能**是出栈序列的是（　）。A. `5,4,3,2,1`　B. `2,1,4,5,3`　C. `4,1,5,3,2`　D. `1,3,2,5,4`
3. 在带头结点的单链表中，要在 `p` 所指结点**之前**插入结点 `s`，最省事的做法是（　）。A. 从头找到 `p` 的前驱，再插入，$O(n)$ B. 把 `s` 插到 `p` 之后，再交换两者的数据域，$O(1)$ C. 无法做到 D. 直接 `s->next = p`，$O(1)$
4. 循环队列用「另设计数器」方案时，判满条件是（　）。A. `front == rear`　B. `(rear+1)%M == front`　C. `count == M`　D. `count == M-1`
5. 已知二叉树的中序序列为 `DBEAFC`、后序序列为 `DEBFCA`，则前序序列为（　）。A. `ABDECF`　B. `ABCDEF`　C. `ABDEFC`　D. `ABDCEF`
6. 下列序列中是**最大堆**的是（　）。A. `(90, 70, 80, 60, 50, 30)`　B. `(90, 80, 70, 60, 50, 85)` C. `(90, 70, 80, 85, 50, 30)`　D. `(30, 50, 60, 70, 80, 90)`
7. 森林中有n 个结点、k 棵树，转换成的二叉树中，从根出发沿右链走到底能经过（　）个结点。A. 1　B. k　C. n　D. $n-k$
8. 使用路径压缩的并查集，路径压缩发生在（　）。A. `Union` 时　B. `Find` 时　C. 初始化时　D. 每次访问数组时

三、辨析与简答题（每题 6 分，共 24 分）

1. 辨析：「用大O 表示时间复杂度，说的就是算法在最坏情况下的代价。」

2. 简答：链式队列在**出队使队列变空**时，为什么必须同时修改队尾指针？不改会出什么问题?
3. 简答：KMP算法中，为什么目标串的下标可以自始至终不回退?
4. 辨析：「二叉搜索树的中序序列有序，所以堆的中序序列也有序。」

四、算法设计与分析题（每题 10 分，共 30 分）

1. 设计算法判断一棵二叉树是否为**完全二叉树**。写明基本思想，给出算法与复杂度。

2. 给定一个长为n 的字符串S，设计$O(n)$ 算法求S 的**最长回文前缀**（即最长的、本身是回文的前缀）。写明思想、算法与代价。

3. 某班有n 名学生，现给出若干条「x 与y 在同一个小组」的信息。设计数据结构与算法，支持：(1) 加入一条信息；(2) 查询两名学生是否同组；(3) 统计小组个数。要求每次操作接近$O(1)$，并说明依据。

五、证明题（10 分）

设一棵非空二叉树有n 个结点。证明：它的**扩充二叉树**（把每棵空子树替换为一个外部结点后得到的树）恰有$n+1$ 个外部结点，并由此说明：n 个结点的二叉树用二叉链表存储时，有$n+1$ 个空指针域。